

Guía de uso del evaluador de expresiones

Luciano David Duarte

ALP 2025 - Licenciatura en Ciencias de la Computación

1 Objetivo de la guía

Esta guía explica cómo compilar, ejecutar y probar el evaluador de expresiones matemáticas desarrollado para el trabajo final. El programa permite escribir expresiones en una variable principal `x`, evaluarlas en un punto y calcular su derivada usando números duales.

El proyecto también incluye procesamiento de archivos, manejo explícito de errores, optimizaciones conservadoras y una API interna que permite evaluar con entornos de variables.

2 Requisitos

Para usar el proyecto se necesita tener instalado:

- GHC compatible con el proyecto;
- Cabal;
- una terminal de Windows, PowerShell o WSL.

Los comandos principales de esta guía están escritos para PowerShell desde la raíz del repositorio:

```
cd C:\Users\lucho\ALP-2025-LCC
```

3 Compilación

Para compilar biblioteca, ejecutable y tests:

```
cabal build all
```

Si Cabal responde `Up to date`, significa que el proyecto ya está compilado con los cambios actuales.

4 Modo interactivo

El modo interactivo permite ingresar una expresión, pedir un valor para `x` y obtener el valor de la función junto con su derivada.

```
cabal run ALP2025-LCC
```

Ejemplo de uso:

```
>>> sin(x) + x^2
>> Ingrese el valor de x: 1
f(1) = 1.8414709848078965
f'(1) = 2.5403023058681398
```

Para salir del modo interactivo se puede escribir:

```
salir
quit
exit
```

5 Procesamiento de archivos

El programa puede leer varias expresiones desde un archivo. Cada línea útil debe tener este formato:

```
expresion @ valor_de_x
```

Ejemplo:

```
x^2 + 2*x + 1 @ 3
sin(x) @ pi/2
log(x) + pi @ e
```

Para procesar un archivo:

```
cabal run ALP2025-LCC -- -f examples\basico.txt
```

También se puede usar la opción larga:

```
cabal run ALP2025-LCC -- --file examples\basico.txt
```

En algunas instalaciones de Windows puede bloquearse el ejecutable generado por Cabal. En ese caso se puede correr el mismo flujo con `runghc`:

```
cabal exec -- runghc -isrc -iapp app\Main.hs -f examples\defensa.txt
```

6 Comentarios en archivos

Los archivos aceptan comentarios de línea:

```
-- Esto es un comentario
x^2 @ 3 -- comentario al final de la línea
```

También aceptan comentarios de bloque:

```
{- Comentario
  de varias líneas -}

(x + {- comentario interno -} 1) * 2 @ 3
```

7 Sintaxis soportada

7.1 Operadores

Operador	Significado
+	suma
-	resta y menos unario
*	multiplicación
/	división
^	potencia

La precedencia usada por el parser es:

potencia > menos unario > multiplicación/división > suma/resta

Por ejemplo, $-x^2$ se interpreta como $-(x^2)$.

7.2 Funciones

Función	Dominio principal
$\sin(x)$, $\cos(x)$	todo número real
$\tan(x)$	donde $\cos(x)$ no sea cero
$\sinh(x)$, $\cosh(x)$, $\tanh(x)$	todo número real
$\operatorname{arsinh}(x)$	todo número real
$\operatorname{arcosh}(x)$	$x \geq 1$
$\operatorname{artanh}(x)$	$ x < 1$
$\operatorname{sqrt}(x)$	$x \geq 0$
$\log(x)$	$x > 0$
$\exp(x)$	todo número real

7.3 Constantes

Se soportan las constantes:

pi
e

El valor ubicado a la derecha de @ puede ser un número o una expresión constante, por ejemplo:

$\sin(x) @ \pi/2$
 $\log(x) @ e$

8 Optimización

Antes de evaluar, el programa aplica optimizaciones simples. Algunos ejemplos:

$x + 0 \rightarrow x$
 $x * 1 \rightarrow x$
 $x / 1 \rightarrow x$
 $x^1 \rightarrow x$
 $2 + 3 \rightarrow 5$

Las optimizaciones son conservadoras. Esto significa que no deben ocultar errores que el evaluador debería detectar. Por eso no se simplifican automáticamente casos como:

```
x / x -> 1
x * 0 -> 0
x^0   -> 1
1^x   -> 1
```

Por ejemplo, si $x = 0$, entonces x/x debe seguir dando división por cero y x^0 debe respetar el caso especial 0^0 .

9 Manejo de errores

Los errores principales se representan de forma tipada y luego se muestran al usuario con mensajes legibles.

9.1 División por cero

```
1/0 @ 0
```

Salida esperada:

```
[X] Error al evaluar: Division por cero
```

9.2 Error de dominio

```
sqrt(-1) @ 0
```

Salida esperada:

```
[X] Error al evaluar: Error de dominio: sqrt requiere argumento no negativo
```

9.3 Variable no definida

El modo de usuario trabaja con la variable x . Si se usa una variable sin valor en el entorno, se reporta como error:

```
y + 2 @ 1
```

Salida esperada:

```
[X] Error al evaluar: Variable no definida: y
```

10 Archivos de ejemplo

La carpeta `examples/` incluye casos para probar distintas partes del lenguaje:

Archivo	Contenido principal
<code>basico.txt</code>	expresiones iniciales
<code>compuestas.txt</code>	composición de funciones
<code>constantes.txt</code>	uso de π y e
<code>constantes_extendido.txt</code>	constantes en casos más variados
<code>trigonometricas.txt</code>	funciones trigonométricas
<code>hiperbolicas.txt</code>	funciones hiperbólicas
<code>todas_funciones.txt</code>	combinación de funciones soportadas
<code>numeros_negativos.txt</code>	menos unario y negativos
<code>precedencia_operadores.txt</code>	precedencia y asociatividad
<code>optimizaciones.txt</code>	reglas de optimización
<code>casos_error.txt</code>	errores controlados
<code>test_comentarios.txt</code>	comentarios de línea y de bloque
<code>valores_especiales.txt</code>	valores borde simples
<code>anidamiento_profundo.txt</code>	expresiones anidadas
<code>identidades_matematicas.txt</code>	identidades conocidas
<code>nuevas_caracteristicas.txt</code>	funcionalidades agregadas
<code>defensa.txt</code>	casos pensados para explicar el trabajo

11 Pruebas

Para correr la suite completa:

```
cabal test
```

La suite actual contiene 125 casos. Incluye una verificación llamada:

Verificacion de Casos de ejemplo

Esa prueba lee todos los archivos `.txt` de `examples/`, verifica que parseen correctamente y comprueba que la evaluación termine de forma controlada, ya sea con resultado correcto o con un error esperado.

También se puede correr la suite directamente:

```
cabal exec -- runghc -isrc test\TestSuite.hs
```

12 Ejemplo recomendado para defensa

Para mostrar varias decisiones del proyecto en una sola ejecución:

```
cabal exec -- runghc -isrc -iapp app\Main.hs -f examples\defensa.txt
```

Ese archivo muestra:

- parsing y precedencia;
- derivación automática con números duales;
- optimización conservadora;
- errores de dominio controlados;
- constantes como π y e ;
- comentarios de bloque dentro de una expresión.

13 Notas finales

El programa no busca ser un sistema de álgebra computacional completo. Su foco es representar un lenguaje pequeño de expresiones, interpretarlo de forma segura y usar números duales para obtener derivadas automáticamente.